

# ESP32 WROOM 프로그래밍 학습 계획서

기준 회로: [esp32wroom\\_30.pdf](#) / GPIO 매핑: [ESP32\\_GPIO\\_포트매핑.md](#) 개발환경: Arduino IDE (또는 PlatformIO)

## 학습 목표

8가지 하드웨어 기능을 단계별로 학습한 후, 최종적으로 **ESP32 Web Server**를 통해 모든 기능을 통합 제어하는 Web App을 완성한다.

## Phase 1: 기본 출력 제어

### Lesson 1 — LED 제어 (RED, YELLOW, GREEN)

| 항목   | 내용                                            |
|------|-----------------------------------------------|
| GPIO | GPIO25 (RED), GPIO26 (YELLOW), GPIO27 (GREEN) |
| 회로   | 각 470Ω 직렬 저항 → LED → GND                      |
| 목표   | digitalWrite, 점멸 패턴, PWM 밝기 제어                |

#### 학습 내용:

- 1. `pinMode()`, `digitalWrite()` — LED ON/OFF
- 2. `delay()` vs `millis()` — 블로킹/논블로킹 점멸
- 3. `ledcSetup()`, `ledcWrite()` — PWM으로 밝기 조절 (0~255)
- 4. 신호등 패턴 만들기 (RED → YELLOW → GREEN 순환)

#### 실습 코드 구조:

```
lesson_01_led/  
├── led_blink.ino      # 기본 점멸  
├── led_pwm.ino        # PWM 밝기 제어  
└── led_traffic.ino    # 신호등 패턴
```

### Lesson 2 — Beep 알람 (BZ2)

| 항목   | 내용                     |
|------|------------------------|
| GPIO | GPIO14 (BZ2 Beep)      |
| 회로   | GPIO14 → BZ2 → GND     |
| 목표   | tone 출력, 주파수 변경, 알람 패턴 |

**학습 내용:**

1. `ledcSetup()`, `ledcAttachPin()`, `ledcWriteTone()` — ESP32 tone 생성
2. 단일 주파수 비프음 출력
3. 알람 패턴 (경고음: 짧은 비프 3회, 긴 비프 1회 등)
4. 주파수 스위프 (저음 → 고음)

**실습 코드 구조:**

```
lesson_02_beep/
├── beep_basic.ino      # 기본 비프음
├── beep_alarm.ino     # 알람 패턴
└── beep_sweep.ino     # 주파수 스위프
```

**Lesson 3 — Buzzer 멜로디 (BZ1)**

| 항목          | 내용                  |
|-------------|---------------------|
| <b>GPIO</b> | GPIO33 (BZ1 Buzzer) |
| <b>회로</b>   | GPIO33 → BZ1 → GND  |
| <b>목표</b>   | 음계 연주, 멜로디 재생       |

**학습 내용:**

1. 음계 주파수 정의 (C4=262Hz, D4=294Hz, ... B4=494Hz)
2. `pitches.h` 활용 — 음계 상수 정의
3. 음표 배열 + 박자 배열로 멜로디 구성
4. 학교종, 생일 축하 등 멜로디 재생
5. Lesson 2 (Beep) + Lesson 3 (Melody) 구분 활용

**실습 코드 구조:**

```
lesson_03_melody/
├── pitches.h           # 음계 주파수 정의
├── melody_scale.ino   # 도레미파솔라시도
├── melody_school.ino  # 학교종 멜로디
└── melody_birthday.ino # 생일 축하 멜로디
```

**Lesson 4 — WS2812 RGB LED Strip**

| 항목           | 내용                           |
|--------------|------------------------------|
| <b>GPIO</b>  | GPIO15 (DIN)                 |
| <b>라이브러리</b> | Adafruit NeoPixel 또는 FastLED |

| 항목 | 내용                     |
|----|------------------------|
| 목표 | 개별 LED 색상 제어, 애니메이션 효과 |

학습 내용:

- 1. NeoPixel 라이브러리 설치 및 초기화
- 2. `setPixelColor(n, r, g, b)` — 개별 LED R/G/B 제어
- 3. 전체 단색 (빨강, 초록, 파랑) 점등
- 4. Rainbow, Chase, Breathing 애니메이션
- 5. GPIO15 부트 스트래핑 핀 주의사항

실습 코드 구조:

```
lesson_04_ws2812/  
├─ ws2812_basic.ino      # R, G, B 단색 제어  
├─ ws2812_rainbow.ino    # 무지개 효과  
└─ ws2812_animation.ino # Chase, Breathing 애니메이션
```

주의사항:

- GPIO15는 ESP32 부트 스트래핑 핀 — 부팅 시 HIGH 유지 필요
- WS2812는 5V 구동이나 3.3V 데이터로도 동작 가능 (레벨 시프터 권장)

Phase 2: 센서 및 통신

Lesson 5 — I2C 버스 디바이스 스캔

| 항목   | 내용                          |
|------|-----------------------------|
| GPIO | GPIO22 (SCL), GPIO21 (SDA)  |
| 회로   | R5, R6 각 10KΩ 풀업            |
| 목표   | I2C 버스 원리 이해, 연결 디바이스 주소 검색 |

학습 내용:

- 1. `Wire.begin(SDA, SCL)` — I2C 초기화
- 2. I2C 주소 스캔 (0x00 ~ 0x7F)
- 3. 발견 디바이스 주소 시리얼 출력
- 4. 예상 주소: OLED (0x3C), AHT20 (0x38)
- 5. I2C 프로토콜 기본 (START, STOP, ACK/NACK)

실습 코드 구조:

```
lesson_05_i2c_scan/  
└─ i2c_scanner.ino      # I2C 디바이스 스캐너
```

예상 출력:

```
I2C Scanner Starting...
Found device at 0x38 (AHT20)
Found device at 0x3C (OLED SSD1306)
Scan complete. 2 devices found.
```

Lesson 6 — AHT20 온습도 센서

| 항목     | 내용                                  |
|--------|-------------------------------------|
| GPIO   | GPIO22 (SCL), GPIO21 (SDA) — I2C 공유 |
| 커넥터    | J4 (aht20)                          |
| I2C 주소 | 0x38                                |
| 라이브러리  | Adafruit AHTX0                      |
| 목표     | 온도/습도 읽기, 주기적 측정                    |

학습 내용:

- 1. AHT20 라이브러리 설치 및 초기화
- 2. `aht.getEvent()` — 온도(°C), 습도(%) 읽기
- 3. 시리얼 모니터에 주기적 출력 (2초 간격)
- 4. 온도 임계값 초과 시 LED 경고 (Lesson 1 연계)
- 5. 습도 임계값 초과 시 Beep 경고 (Lesson 2 연계)

실습 코드 구조:

```
lesson_06_aht20/
├─ aht20_basic.ino      # 기본 온습도 읽기
└─ aht20_alarm.ino     # 임계값 경고 (LED + Beep 연계)
```

Lesson 7 — RTC (Real Time Clock)

| 항목    | 내용                                       |
|-------|------------------------------------------|
| 통신    | I2C (GPIO22/21 공유) 또는 ESP32 내부 RTC       |
| 라이브러리 | ESP32Time (내부 RTC) 또는 RTCLib (외부 DS3231) |
| 목표    | 시간 설정/읽기, NTP 동기화                        |

학습 내용:

1. ESP32 내부 RTC 사용 — `ESP32Time` 라이브러리
2. 시간 설정: `setTime(초, 분, 시, 일, 월, 년)`
3. 시간 읽기: `getTime()`, `getDate()`
4. **NTP 서버 동기화** (WiFi 필요)
  - `configTime(gmtOffset, daylightOffset, ntpServer)`
  - 한국 시간대: GMT+9 (32400초)
5. 시간 기반 알람: 특정 시각에 Buzzer 울리기

#### 실습 코드 구조:

```
lesson_07_rtc/
├── rtc_internal.ino      # 내부 RTC 시간 설정/읽기
├── rtc_ntp_sync.ino      # NTP 시간 동기화
└── rtc_alarm.ino        # 시간 기반 알람
```

## Lesson 8 — UART 입력 및 출력

| 항목          | 내용                                 |
|-------------|------------------------------------|
| <b>GPIO</b> | GPIO17 (TX2), GPIO16 (RX2) — UART2 |
| <b>커넥터</b>  | J5 (Lora_con)                      |
| <b>목표</b>   | 시리얼 통신, 명령어 파싱, LoRa 모듈 통신 기초      |

#### 학습 내용:

1. `Serial` (USB) vs `Serial2` (UART2) 차이
2. `Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2)` — UART2 초기화
3. Serial Monitor → ESP32 → Serial2 데이터 전달 (브릿지)
4. 문자열 명령어 파싱 (`LED ON`, `BEEP`, `TEMP?` 등)
5. LoRa 모듈 AT 명령어 전송 기초

#### 실습 코드 구조:

```
lesson_08_uart/
├── uart_echo.ino        # Serial2 에코 테스트
├── uart_bridge.ino      # USB ↔ UART2 브릿지
└── uart_command.ino    # 명령어 파싱 제어
```

#### 명령어 예시:

```
> LED RED ON      → GPIO25 HIGH
> LED ALL OFF     → 전체 LED OFF
> BEEP 3          → 비프음 3회
> MELODY PLAY     → 멜로디 재생
```

|         |            |
|---------|------------|
| > TEMP? | → 현재 온도 응답 |
| > TIME? | → 현재 시간 응답 |

## Phase 3: Web App 통합

### Lesson 9 — ESP32 Web Server 구축

| 항목    | 내용                                          |
|-------|---------------------------------------------|
| 라이브러리 | WiFi.h, WebServer.h, ESPAsyncWebServer (권장) |
| 목표    | Lesson 1~8 전체 기능을 웹에서 제어                    |

학습 내용:

- 1. WiFi STA 모드 연결
- 2. ESPAsyncWebServer + AsyncWebSocket 설정
- 3. HTML/CSS/JS 프론트엔드 (SPIFFS 또는 LittleFS)
- 4. REST API 설계 + WebSocket 실시간 데이터

### Web App 기능 구성

| 기능        | API Endpoint  | Method   | 설명                                            |
|-----------|---------------|----------|-----------------------------------------------|
| LED 제어    | /api/led      | POST     | {"color":"red","state":"on","brightness":128} |
| Beep 알람   | /api/beep     | POST     | {"pattern":"alarm","count":3}                 |
| Melody 재생 | /api/melody   | POST     | {"song":"birthday"}                           |
| WS2812 제어 | /api/ws2812   | POST     | {"mode":"rainbow","r":255,"g":0,"b":0}        |
| I2C 스캔    | /api/i2c/scan | GET      | 연결 디바이스 목록 반환                                 |
| 온습도       | /api/sensor   | GET      | {"temp":25.3,"humidity":60.1}                 |
| RTC 시간    | /api/time     | GET/POST | 시간 조회/설정                                      |
| UART 전송   | /api/uart     | POST     | {"message":"AT+SEND=hello"}                   |

### Web App UI 구성

| ESP32 Control Dashboard |                                                                                     |                             |                                     |
|-------------------------|-------------------------------------------------------------------------------------|-----------------------------|-------------------------------------|
| LED 제어                  |  | RED                         | [ON] [OFF] <input type="checkbox"/> |
|                         |  | YELLOW                      | [ON] [OFF] <input type="checkbox"/> |
|                         |  | GREEN                       | [ON] [OFF] <input type="checkbox"/> |
| WS2812                  | [R <input type="checkbox"/>                                                         | [G <input type="checkbox"/> | [B <input type="checkbox"/>         |

|                 |                                                                                                   |
|-----------------|---------------------------------------------------------------------------------------------------|
| RGB LED         | Mode: [단색 무지개 Chase]                                                                              |
| Sound           | [Beep] [Alarm] [Melody ▼]                                                                         |
| Sensor<br>(실시간) | <div><div><div>🌡 25.3°C</div><div>💧 60.1%</div></div><div><div></div><div>(그래프)</div></div></div> |
| I2C             | [Scan] → 0x38, 0x3C 발견                                                                            |
| RTC 시간          | 2026-03-26 14:30:00 [동기화]                                                                         |
| UART<br>터미널     | [입력창] [Send]<br>> 수신 데이터 표시...                                                                    |

Web App 프로젝트 구조

```
lesson_09_webapp/  
├── src/  
│   ├── main.cpp           # 메인 (WiFi, 서버, 하드웨어 초기화)  
│   ├── led_control.h/cpp   # LED 제어 모듈  
│   ├── sound_control.h/cpp # Beep + Melody 모듈  
│   ├── ws2812_control.h/cpp # WS2812 모듈  
│   ├── sensor_control.h/cpp # AHT20 + I2C 모듈  
│   ├── rtc_control.h/cpp   # RTC 모듈  
│   └── uart_control.h/cpp  # UART 모듈  
├── data/  
│   ├── index.html         # 메인 대시보드  
│   ├── style.css          # 스타일  
│   └── app.js             # 프론트엔드 로직 + WebSocket  
└── platformio.ini         # PlatformIO 설정
```

학습 일정 (권장)

| 일차  | Lesson | 내용                  | 난이도   |
|-----|--------|---------------------|-------|
| 1일차 | 1      | LED 제어 (GPIO 출력 기초) | ★☆☆☆☆ |
| 2일차 | 2      | Beep 알람 (톤 출력)      | ★★☆☆☆ |
| 2일차 | 3      | Buzzer 멜로디 (음계 배열)  | ★★☆☆☆ |
| 3일차 | 4      | WS2812 RGB LED      | ★★★☆☆ |
| 4일차 | 5      | I2C 스캔 (통신 기초)      | ★★☆☆☆ |
| 4일차 | 6      | AHT20 온습도 센서        | ★★☆☆☆ |
| 5일차 | 7      | RTC + NTP 동기화       | ★★★☆☆ |

| 일차    | Lesson | 내용         | 난이도     |
|-------|--------|------------|---------|
| 5일차   | 8      | UART 통신    | ★★★★☆☆  |
| 6~8일차 | 9      | Web App 통합 | ★★★★★☆☆ |

필요 라이브러리

| 라이브러리             | 용도                   | 설치              |
|-------------------|----------------------|-----------------|
| Adafruit NeoPixel | WS2812 제어            | Library Manager |
| Adafruit AHTX0    | AHT20 센서             | Library Manager |
| ESP32Time         | 내부 RTC               | Library Manager |
| ESPAsyncWebServer | 비동기 웹서버              | GitHub          |
| AsyncTCP          | ESPAsyncWebServer 의존 | GitHub          |
| ArduinoJson       | JSON 처리              | Library Manager |
| LittleFS          | 파일시스템                | ESP32 내장        |

핀 배정 요약 (전체 프로젝트)

| GPIO   | 용도                | Phase |
|--------|-------------------|-------|
| GPIO25 | RED LED           | 1     |
| GPIO26 | YELLOW LED        | 1     |
| GPIO27 | GREEN LED         | 1     |
| GPIO14 | BZ2 Beep          | 1     |
| GPIO33 | BZ1 Buzzer Melody | 1     |
| GPIO15 | WS2812 DIN        | 1     |
| GPIO22 | I2C SCL           | 2     |
| GPIO21 | I2C SDA           | 2     |
| GPIO17 | UART2 TX          | 2     |
| GPIO16 | UART2 RX          | 2     |